Orrery

REFERENCE

Orrery 0.6
Syntax reference

Every statement, shape, anchor, connection, style property and motion rule in the language, with examples that compile as written.

1	At a glance
2	Files, comments and names
3	Values, units and expressions
4	Parts, groups, repeat and define
5	Shapes and anchors
6	Connections
7	Linkages and closed loops
8	Style
9	Motion
10	Camera, lights and scene
11	Complete examples: V8 engine, steam locomotive
12	Diagnostics
13	Tools for people and AI
A	Quick reference card
B	The AI prompt

Conventions in this guide: `monospace` *is code; italic names a placeholder; a property marked* **bold** *in a table is required.*

1 At a glance

Orrery describes a mechanism the way CSS describes a page: what exists, how it looks, and how it changes. You declare parts and how they connect. The compiler places them, solves joints, gear trains and closed loops at every frame, and renders a GIF.

Rule	In practice
A part declares what it connects to	Inside its own block: `attach hinge slider mesh couple belt pin`.
Units decide what a value means	`90deg` is an angle, `90deg/s` a spin rate, `90deg/s2` an angular acceleration. The same goes for `mm`, `mm/s` and `mm/s2`.
Values come first, then keywords	`rotate: 60deg/s about G1.axis; orbit: Sun 30deg/s; follow: Car offset (…);`
Drive only what starts the motion	Anything hinged, meshed, coupled, belted or pinned to a driver follows by itself.

A complete project

Two gears and a handle · .scene

```
Big::Gear   { teeth: 30; module: 2mm; width: 10mm; at: (0, 0, 0); }
Small::Gear { teeth: 15; module: 2mm; width: 10mm; mesh: Big; }
Handle::Rod { length: 40mm; radius: 2.5mm; attach: end.a to Big.face.front; }

Observer { position: (20mm, -30mm, 180mm); look-at: (22mm, 0, 0); }
Scene    { duration: 6s; }
```

Two gears and a handle · .style

```
Gear   { color: steel; material: metal; outline: 1px #1e252c; }
Small  { color: brass; label: "2x speed"; }
Handle { color: #b8442e; }
```

Two gears and a handle · .motion

```
Handle { forever: rotate: 60deg/s about Big.axis; }
```

The handle is attached to the big gear, so turning it turns the gear; `mesh` *places the small gear at the right distance, aligns the teeth and sets the ratio to −2.*

2 Files, comments and names

File	Answers	Contains
`.scene`	What exists?	let and define statements, parts and groups, connections, Observer, Light, Scene
`.style`	How does it look?	Selector blocks with colour, material, outline, opacity, label and trail
`.motion`	How does it move?	Rules with time blocks that drive parts, joints and the camera

Item	Rule	Example
Comment	// to the end of the line; /* … */ for blocks (as in CSS)	`// a note /* a note */`
Colour or id	# followed directly by letters or digits (never a comment)	`#e0a63a #G2`
Part name	Starts with a capital letter; case-sensitive	`Crank, FL, Door`
let name, parameter	Starts with a lower-case letter	`wheel, cruise, mount`
Property	name: value; Every property ends with a semicolon	`radius: 40mm;`

Item	Rule	Example
Reserved	Not usable as let names	`t i u v x y z axis to on at about step offset`

One file or three

The same project can be a single `.orrery` file in which a line `@scene`, `@style` or `@motion` starts each section. It means exactly the same as the three files, and is the easiest form to review or to hand to an AI assistant. The studio switches between the two with **Split / Combined**; the command line converts with `join` and `split`.

project.orrery

```
@scene
Crank::Link { length: 40mm; hinge: end.a on (0, 0, 0); }

@style
Crank { color: #b8442e; }

@motion
Crank { forever: rotate: 180deg/s; }
```

Each marker stands alone on its line (a comment may follow) and each section appears once. Problems in a single file are reported by their line in that file, with the section they belong to. E0105: text before the first marker; E0106: unknown section; E0107: a section repeated.

3 Values, units and expressions

Units

Quantity	Position or angle	Speed	Acceleration
Length	`mm cm m`	`mm/s m/s`	`mm/s2 m/s2`
Angle	`deg rad turn`	`deg/s turn/s`	`deg/s2`
Time	`s ms`		
Fraction	`%`		

Dividing by a time also makes a rate: `360deg / 12s` *is a spin rate. A bare number is read as millimetres for lengths and seconds for times; writing units everywhere is clearer.*

Kinds of value

Kind	Example	Notes
Number	`40mm 90deg 0.8`	No space between number and unit
Vector	`(0, 1500mm, 0)`	x right, y up, z towards the viewer
Anchor	`Door.panel.edge.left`	A point or line on a part (section 5)
Point maths	`Frame.face.top + (0, 10mm, 0)`	An anchor plus or minus a vector
Range	`0deg..110deg`	Limits and time blocks
Change	`0deg -> 90deg ease-in-out`	From one value to another across a time block
Colour	`#e0a63a steel peru`	Hex, CSS names, or a named finish
Text	`"500 mm/s"`	Labels

Expressions

Arithmetic `+ - * /` and these functions work anywhere a number does: `sin cos tan sqrt abs min max pow floor exp atan2 length`. `PI` is π. In motion values, `t` is time in seconds.

let and expressions · .scene

```
let year = 12s;
let cruise = (600mm/s, 0, 0);
Earth::Sphere { radius: 10mm; at: (140mm * cos(250deg), 0, 140mm * sin(250deg)); }
```

4 Parts, groups, repeat and define

Declaring a part: `Name::Type.class1.class2 { property: value; … }`. Classes are optional and shared by any number of parts.

Placement

Property	Example	Meaning
`at`	`at: Frame.face.top + (0, 10mm, 0);`	Where the part's centre goes
`align`	`align: face.bottom;`	Put this anchor on the at point instead of the centre
`orient`	`orient: 90deg about x;`	Starting rotation about x, y, z or a vector
`visible`	`visible: false;`	Hidden but still usable, for example as a path
`repeat`	`repeat: 11 step (500mm, 0, 0);`	Numbered copies (see below)
`scatter`	`scatter: 40 in plane xz (600mm, 400mm) seed 7 spin 360deg vary 20% apart;`	Random copies, the same every build: in plane (xz, "x + y + z = 300mm", normal (1, 1, 0)), along, in box, in disc, on circle, in sphere

Groups

A `Group` holds named members that share one frame and move as one. Members can be placed against each other. Refer to a member as `Door.knob` and to its anchors as `Door.panel.edge.left`. `anchor` adds a named point.

A group · .scene

```
Door::Group.wooden {
    panel: Box    { size: (800mm, 2000mm, 40mm); }
    knob:  Sphere { radius: 30mm; at: panel.face.front + (320mm, 0, 25mm); }
    anchor grip: knob.center;
    at: (430mm, 1000mm, 0);
}
```

repeat and i

`repeat: N` makes copies named *Name*1 to *Name*N. `step (dx, dy, dz)` shifts each copy; `i` (0, 1, 2 …) can be used in any expression of the copy.

repeat with step and with i · .scene

```
Road::Group {
    surface: Box { size: (6000mm, 4mm, 700mm); }
    dash: Box.dash { size: (220mm, 2mm, 18mm); at: surface.face.top + (-2500mm, 1mm, 0); repeat: 11 step (500mm, 0, 0); }
}
Trunk::Cylinder.trunk { radius: 9mm; height: 70mm; repeat: 7; at: (680mm * cos(i * 0.9), 35mm, 620mm * sin(i * 0.9)); }
```

scatter: random copies · .scene

```
Leaf::Polygon.leaf { sides: 5; radius: 22mm; thickness: 2mm; scatter: 40 in plane xz (640mm, 440mm) seed 4 spin 360deg vary 30% apart; }
```

define: your own part types

`define Name(params) :: Type { … }` makes a reusable type built on a shape or a Group. Parameters can have defaults and can hold any value, including an anchor. Use the type with values by position or by name; properties in the part's own block are added and replace any with the same name. The type name is also a style selector and a motion target.

define · .scene

```
define Tyre(mount, r = 55mm) :: Wheel { radius: r; width: 34mm; hinge: axis on mount; }

FL::Tyre(Car.axle1.end.b);
RL::Tyre(mount: Car.axle2.end.b, r: 60mm);
```

Every Tyre at once · .style

```
Tyre { color: gradient(#16181b, #5d6670, axis: x); }
```

Every Tyre at once · .motion

```
Tyre { forever: roll: Road; }
```

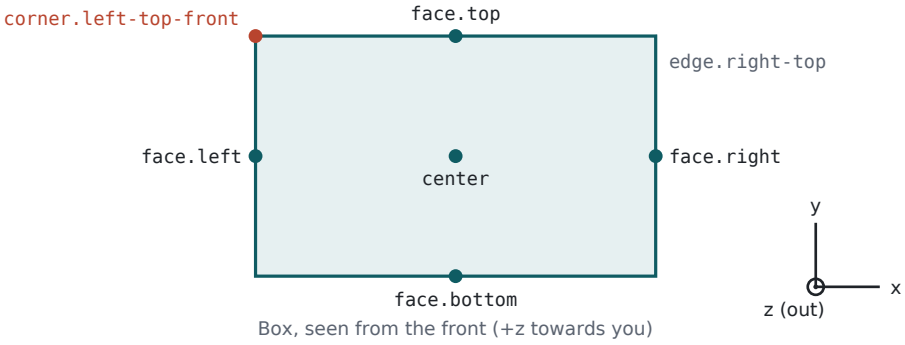
Parameters hide let values of the same name. A type built on a Group cannot be placed inside another group.

5 Shapes and anchors

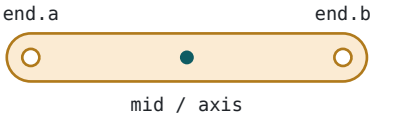
Anchors are named points and lines in a part's own frame. They move with the part, so you can place and connect parts without computing coordinates. Every shape has `center` and `com`, its true centre of mass for a uniform solid (long name `center-of-mass`), with `com.x`, `com.y` and `com.z` as lines through it: `hinge: com.z on Base.axis` pivots a part at its balance point. A group's com weighs members by volume.

Type	Properties	Anchors
Box	**size** (x, y, z)	face.*, edge.*face*, edge.*f1-f2*, corner.*x-y-z*, axis (y)
Sphere	**radius**	pole.top, pole.bottom, axis (y)
Cylinder	**radius**, **height**	axis (y), face.top, face.bottom
Cone	**radius**, **height**	axis, face.top, face.bottom, apex
Torus	**radius**, tube	axis (z), face.front, face.back
Rod	**length**, radius	axis (y), end.a (bottom), end.b (top): lines along y
Link	**length**, width, thickness	end.a, end.b, mid, axis: lines along z (pin axles); line (along x)
Gear	**teeth**, module (2mm), width	axis (z), face.front, face.back, pitch-circle
Pulley, Wheel	**radius**, width	axis (z), face.front, face.back
Disc	**radius**, thickness	axis (z), normal, face.front, face.back
Rect	**size** (w, h), thickness	like Box, plus normal
Spring	**radius**, **length**, turns, wire	axis (y), end.a, end.b
Line	**from**, **to**, stroke	start, end, mid, axis
Arrow	**from**, **to**, stroke, head	tail, head, start, end, mid
Curve	**domain**, **point**, stroke	start, mid, end
Tube	**radius**, inner, **height**	axis (y), face.top, face.bottom
Capsule	**radius**, **length**	axis (y), end.a, end.b
Prism, Pyramid	sides, **radius**, **height**	axis (y), face.top, face.bottom (Pyramid: apex)
Dome	**radius**	axis (y), face.bottom, pole.top
Wedge	**size** (x, y, z)	like Box, plus slope
Plate	**size** (w, h), thickness, corner, hole	like Box, plus normal, hole
Beam	**length**, height, width, thickness	I-beam along x: like Box, plus end.a, end.b, line
Ring, Nut	**radius**, inner or hole, thickness	axis (z), face.front, face.back

Type	Properties	Anchors
Bolt	**radius**, **length**, head	axis (y), end.a, end.b, head
Cam	**radius**, lift, width	axis (z), face.front, face.back, nose
Star	points, **radius**, inner, thickness	axis (z), face.front, face.back
Mesh	**src** "file.stl/.glb", **size**, fit	like Box: your own 3D model, fitted in the size
Triangle, Ellipse	**size** (w, h), apex (Triangle), thickness	2D, facing +z: like Box, plus normal
Polygon	sides, **radius**, thickness	2D: axis (z), normal, face.front, face.back
Sector	**radius**, angle, start, inner, thickness	2D pie slice or ring segment: tip, mid, edge.start, edge.end
Arc	**radius**, angle, start, stroke	a curved line: start, mid, end, axis
Path	**points** (x, y) (x, y) …, thickness	2D outline: axis, normal, start, end
Text	**text** "…", size, facing: front | camera	words as a part: like Box, plus normal
Plot	**size**, function (of x, t) or data (x, y) …, kind, x-axis, y-axis, reveal	a chart: like Box, plus origin, end
Voice	voice, rate, pitch, volume, captions	a narrator for say: lines (not a part)
Group	members	center, com, com.x/.y/.z, member anchors, custom anchors



Box, seen from the front (+z towards you)



Link, lying along x. end.a, end.b and mid are lines along z: pin axles.

Form	Kind	Example
face.f	point with a normal (also usable as a line)	`Frame.face.top`
edge.f	line through the face centre, along its longer side	`Door.panel.edge.left`
edge.f1-f2	line where two faces meet	`Base.edge.right-top`
corner.x-y-z	point (any order)	`Box.corner.left-top-front`
axis	line	`G1.axis`
end.a, end.b	line (Rod: along y; Link: along z)	`Crank.end.b`
anchor name: …;	custom point	`anchor tip: (0, 120mm, 0);`

Faces: front (+z), back (−z), left (−x), right (+x), top (+y), bottom (−y). Anchor names are never guessed: a wrong one is error E0310, which lists the valid ones.

Curves: shapes as functions

A curve that changes with time · .scene

```
Wave::Curve {
    domain: u in 0..1;
    point: (u * 400mm - 200mm, 40mm * sin(6.283 * (u * 2 - t / 2s)), 0);
    stroke: 4mm;
}
```

6 Connections

A connection is written inside the block of the part that moves. It places the part for you and keeps it connected while things move.

Property	Freedom	Forms

`attach`	none: moves as one	`attach: own-anchor to Other.anchor;`
		`attach: Other.anchor;`
		`attach: end.a to Flywheel.face.front + (0, 0, 10mm);`

`hinge`	turns about a line, rides on Other	`hinge: own-line on Other.line;`
		`hinge: Other.line;`
		`hinge: end.a on (0, 0, 0);`

`slider`	slides along a line	`slider: axis on Barrel.axis;`
`range`	limits a hinge or slider	`range: -45deg..45deg;`
`mesh`	gear ratio from teeth (−z1/z2)	`mesh: G1;` `mesh: G1 at 60deg;`
`couple`	any ratio between round parts	`couple: MinuteHub 1/12;`
`belt`	same direction, ratio r1/r2	`belt: Motor;`
`pin`	two points kept together	`pin: own-point to Other.point;` `pin: Other.point;`

attach

Moves the part so its anchor sits on the other anchor, then fuses them. With one anchor, the part's centre is used.

hinge and slider

Own line on other line, or just the other line (the part's own `axis` is used). The other side can be a point: an anchor plus an offset, or a fixed position; the part then turns about its own line through it. A hinge carries its part along when the part it sits on moves. A Group is hinged by naming a member's line inside the group block.

A hinged group · .scene

```
Frame::Box { size: (60mm, 2000mm, 60mm); at: (0, 1000mm, 0); }
Door::Group {
    panel: Box { size: (800mm, 2000mm, 40mm); }
    hinge: panel.edge.left on Frame.edge.right;
    range: 0deg..110deg;
}
```

mesh, couple and belt

`mesh` places a gear at the right centre distance, turns it so the teeth interlock and sets the ratio; `at 60deg` places it round the other gear. Meshed gears need the same module. Gear loops whose ratios disagree are error E0412.

Gears and a belt · .scene

```
G1::Gear { teeth: 36; module: 2mm; at: (0, 0, 0); }
G2::Gear { teeth: 12; module: 2mm; mesh: G1; }
G3::Gear { teeth: 18; module: 2mm; mesh: G2 at 60deg; }
Motor::Pulley { radius: 20mm; width: 16mm; at: (0, -150mm, 0); }
Drum::Pulley  { radius: 60mm; width: 16mm; at: (250mm, -150mm, 0); belt: Motor; }
```

7 Linkages and closed loops

In a chain of hinges each part rides on the one before. A closed loop comes back to the ground, so its parts must also stay together. `pin` names the two points that must meet. Drive one joint; every hinge or slider in the loop that has no motion rule is solved at each frame so the pins hold.

Four-bar linkage · .scene

```
Crank::Link   { length: 40mm;  hinge: end.a on (0, 0, 0); }
Coupler::Link { length: 120mm; hinge: end.a on Crank.end.b + (0, 0, 6mm); pin: end.b to Rocker.end.b; }
Rocker::Link  { length: 90mm;  orient: 90deg about z; hinge: end.a on (110mm, 0, 0); }
Scene { duration: 2s; }
```

Four-bar linkage · .motion

```
Crank { forever: rotate: 180deg/s; }
```

Slider-crank · .scene

```
Crank::Link      { length: 50mm; hinge: end.a on (0, 0, 0); }
Bore::Cylinder   { radius: 22mm; height: 220mm; orient: 90deg about z; at: (200mm, 0, 0); }
Piston::Cylinder { radius: 20mm; height: 40mm; slider: axis on Bore.axis; }
Rod::Link        { length: 180mm; hinge: end.a on Crank.end.b + (0, 0, 6mm); pin: end.b to Piston.center; }
Scene { duration: 4s; }
```

Slider-crank · .motion

```
Crank { forever: rotate: 90deg/s; }
```

- **Use Link for bars.** Its ends are lines along z, so hinges and pins on them turn in the xy plane.
- **A pin is an axle.** Parts may sit at different depths along it; `+ (0, 0, 6mm)` layers the coupler in front of the crank.
- **The starting pose picks the branch.** Most four-bars close two ways (elbow up or down). Orient the rocker roughly where you want it.
- **The whole motion is checked when you compile.** E0352: the loop cannot close (the gap is given in mm). E0353: the linkage locks, with the time. E0354: every joint in the loop is driven.

8 Style

Selector	Matches
`Gear`	every part of that type or define, or the part named Gear
`Door`	the part named Door, including all its members
`Door.knob`	one member of a group
`.wooden`	every part with that class (members inherit their group's classes)
`#G2`	the part named G2
`Crank, Knob`	both

Later rules win; there is no specificity scoring.

Property	Values
`use`	names of styles (Metal::Style { … }), left to right: a later style wins; the rule's own properties win over all

Property	Values
`color`	#hex, #rrggbbaa (alpha), rgb(r, g, b), rgba(r, g, b, a), hsl(h, s%, l%), transparent, CSS names, steel brass oak copper aluminium iron rubber walnut glass graphite, or gradient(#a, #b, axis: x). transparent hides the faces and keeps the outline
`material`	matte plastic metal brushed chrome gold glass flat glow
`outline`	1px #2b3138
`opacity`	0 to 1, or 40%. Faces, outline and label fade together; multiplies with colour alpha
`luminosity`	[level] [colour] [range]: the part shines and lights parts near it. 100% is a lamp, up to 400%. Solid parts get a halo; see-through ones add light like a beam. Animatable
`roughness, metalness`	0 to 1 (or %); override the material
`wireframe / shadow`	on | off / none | cast
`text`	"PISTON": printed on the part's faces (\n starts a new line). On a Text part it replaces the scene text
`text-side`	front (default) back left right top bottom, several, or all. Round parts wrap it round their side
`font-family`	sans | serif | mono | "Any Installed Font" (3D letters: sans, serif, mono)
`font-size / -weight`	24mm or 40% of the face (omit to fit) / normal | bold | 100 to 900
`font-style`	normal | italic | 3d (solid letters with depth and shadows) | italic 3d
`text-color / -depth`	colour of the letters (a Text also takes color) / how far 3D letters stand out
`text-align, -padding`	left | center | right; space to the edge of the face or panel
`text-background`	a panel behind the letters, e.g. #10203080; line-height: 1.2;
`label`	a tag above the part, always on top; prefer Text for anything long-lived
`label-color, -size`	the label text colour, and its size (140%)
`trail`	[anchor] [time] [colour], each optional: the path the anchor follows. No time means the whole run so far.

Style rules · .style

```
Gear  { color: steel; material: metal; outline: 1px #1e252c; }
#G2   { color: brass; }
.tyre { color: gradient(#141619, #b9c0c8, axis: x); }
Sun   { color: #ffcf5a; luminosity: 160%; label: "Sun"; }
Pen   { trail: #c2410c; }
Blade { trail: face.top 1s #2f6f8f; }
```

Named styles and use · .style

```
Metal::Style    { material: metal; color: steel; roughness: 0.3; }
Polished::Style { roughness: 0.1; outline: 1px #2b3138; }
Brass::Style    { use: Metal; color: brass; }
Gear  { use: Brass, Polished; }
Crank { use: Metal; color: #8f99a3; }
```

9 Motion

A rule names a target (a part, a `.class`, a define name, several of these separated by commas, or `0bserver`) and lists time blocks.

Block	Runs
`forever:`	the whole animation
`0s..2s:`	during that range
`then:`	after the previous block, for as long as it lasted
`then 3s:`	after the previous block, for 3 s

How values behave over time

Value	Example	Behaviour
Position or angle	`rotate: 90deg;`	Set from the start of its block; the latest block that has started wins
Change	`rotate: 0deg -> 90deg ease-in-out;`	Moves across its block, then holds the end value
Speed	`rotate: 60deg/s;`	Moves during its block; what it moved stays moved; speeds add up
Change of speed	`move: (0, 0, 0) -> (600mm/s, 0, 0);`	Speeds up or slows down across its block; needs a time range
Acceleration	`move: (0, -9.81m/s2, 0);`	Constant during its block
Function of t	`rotate: 35deg * sin(t * 10);`	Any expression of time in seconds

Easings follow CSS: linear (default), ease, ease-in, ease-out, ease-in-out, cubic-bezier(x1, y1, x2, y2).

Properties

Target	Property	Example
free part	`rotate`	`rotate: 60deg/s about G1.axis;`
free part	`orbit`	`orbit: Earth 360deg / 2s;`
free part	`move`	`move: (500mm/s, 0, 0);`
free part	`along`	`along: Run 12s;`
free part	`x, y, z`	`x: 0;`
part or group	`opacity`	`0s..1s: opacity: 0 -> 1 ease-in;`
part or group	`color`	`0s..2s: color: #2f6fb0 -> #e0a63a;`
part or group	`luminosity`	`0s..1s: luminosity: 0 -> 100%;`
hinged part	`rotate`	`rotate: 0deg -> 95deg ease-in-out;`
hinged part	`roll`	`roll: Road;`
sliding part	`move`	`move: -90mm -> 90mm;`
any rule	`say`	`say: Narrator "The valve opens";`

`rotate` *without* `about` *uses the part's own axis (or y).* `orbit` *circles a centre part about the vertical and follows it if it moves, so orbits nest. Several* `move` *lines add up.* `roll` *spins a hinged wheel by the distance its centre travels divided by its radius.*

Acceleration patterns

Speed up and brake · throw · spin up · .motion

```
let cruise = (600mm/s, 0, 0);
Car {
    0s..1.5s: move: (0, 0, 0) -> cruise ease-in;
    then:     move: cruise;
    3s..4.2s: move: cruise -> (0, 0, 0) ease-out;
}
Ball {
    0s..1.5s: move: (600mm/s, 1200mm/s, 0);
              move: (0, -1.62m/s2, 0);
}
Motor {
    0s..1s:  rotate: 360deg/s2;
    then 3s: rotate: 360deg/s;
}
```

10 Camera, lights, scene and sound

Scene settings · .scene

```
Observer { position: (2600mm, 1700mm, 3400mm); look-at: (450mm, 1000mm, 0); zoom: 1; fov: 40deg; }
Light    { kind: directional; direction: (-1, -2, -1.5); power: 0.85; }
Light    { kind: point; at: (0, 0, 0); power: 0.5; }
Scene    { duration: 4.5s; sky: dusk; ground: #2c3328; }
```

A chart, a narrator and a voice-over file · .scene

```
Wave::Plot { size: (420mm, 240mm); function: sin(x - t * 2); x-axis: 0..10 step 2 "time (s)"; reveal: min(1, t / 3s);
    }
Narrator::Voice { voice: en-GB; rate: 95%; }
// a voice-over file, once added with Insert › 3D models and audio:  VoiceOver::Sound { src: "intro.mp3"; }
```

In motion, `say: Narrator "words" [at end];` *speaks with the browser's voice (preview and 3D player) and shows a caption, drawn into exports and saved as .srt. Audio files play with* `sound:` *and are mixed into MP4 and WebM.*

`frame: reel;` *sets the picture: youtube, youtube-short, reel, tiktok, instagram, linkedin, square, slides, 4k, cinema, a ratio (9:16) or pixels (1080x1920). The preview shows the frame and its safe area; exports use its size.*

`sky` *wraps everything in a glowing sphere that travels with the camera: day, dawn, dusk, sunset, night, space, overcast, studio, or up to three colours (top, horizon, below). It tints the ambient light; night and space have stars (*`stars: 1500;`*).* `ground` *lays a floor that fades into the sky.* `background: #070a10;` *and* `grid: floor;` *still work.*

`background: transparent;` *keeps the background clear: PNG frames, GIF, a saved PNG and the 3D player keep the transparency; MP4 and WebM fill it with the theme colour.*

Without an Observer the whole scene is framed. Without a duration, the end of the last time block is used; if every block runs forever, 6 s; with no motion, 4 s. Light power runs from 0 to 1.

Camera property	Example	Notes
`follow`	`follow: Car offset (380mm, 260mm, 1150mm);`	Keeps an offset and looks at the part
`orbit`	`orbit: Mouse 60deg/s;`	Circles a part, even a moving one
`look-at`	`look-at: Door.knob;`	Faces a part or point
`zoom`	`zoom: 1 -> 2;`	Zooms about the look-at point
`move, x, y, z, position`	`move: (0, -20mm/s, 0);`	Combine with follow or orbit

Sound

Declare a Sound in the scene and play it from any motion rule. Sounds are generated by the studio from recipes, so they are free of copyright; they follow the motion, so they stay in time when it changes.

A dropped ball, ticks, then a bell · .scene

```
Ball::Sphere { radius: 30mm; at: (0, 200mm, 0); }
Knock::Sound { kind: knock; }
Tick::Sound  { kind: tick; volume: 60%; }
Bell::Sound  { kind: bell; pitch: 120%; }
```

A dropped ball, ticks, then a bell · .motion

```
Ball {
    0s..1.2s: move: (0, 0, 0) -> (0, -200mm, 0) ease-in;
              sound: Knock at end;      // lands
    then 4s:  sound: Tick every 1s 50%;
}
Scene { sound: Bell at 5s; }
```

Write	Plays
`sound: Knock;`	when the time block starts
`sound: Knock at end;`	when the block ends (a part landing)
`sound: Knock at 1.5s;`	that long after the block starts
`sound: Tick every 1s;`	repeatedly while the block runs
`... 60%`	quieter (after any of the above)

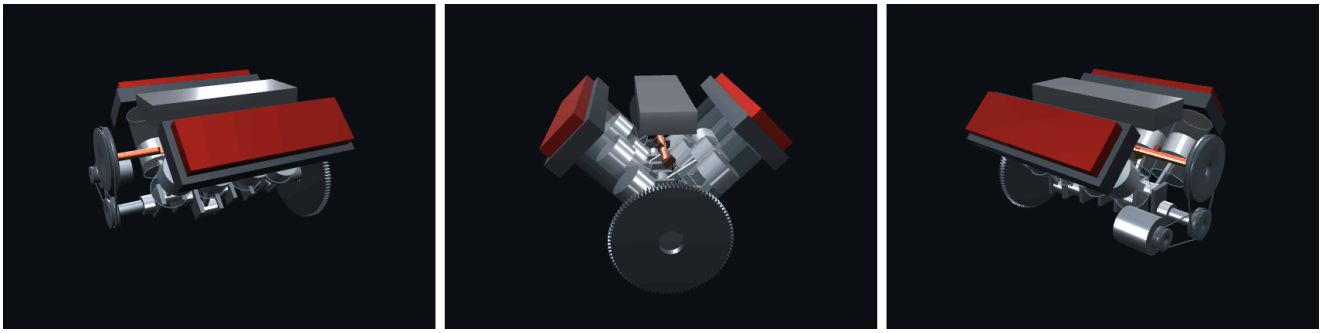
Kinds: `tick tock ratchet clank latch gear spring rattle` · `footstep clang chug hiss rumble` · `knock thud click snap pop tap` · `whoosh swish whirr servo slide creak squeak` · `chime bell ding beep blip success alert` · `hum`. *Optional:* `pitch` *(50% to 200%),* `volume`*, and* `length` *for the movement kinds and hum. MP4 and WebM exports carry the sound;* `render -o sound.wav` *writes it alone.*

11 Complete examples

These are the studio's examples, unchanged. Export them all as folders with `orrery examples out/`.

V8 engine: the default example

A cross-plane V8, cut away. One line turns the crankshaft; eight connecting rods and eight pistons are closed loops solved together at every frame; a belt turns the gold camshaft at half speed and spins the alternator; chrome, brushed and gold materials reflect a soft studio light; and the camera makes one slow lap.



Three frames of the exported GIF.

V8 · .motion

```
// One motion rule drives the whole engine.
Crank    { forever: rotate: 270deg/s about z; }

// One slow lap around it.
Observer { forever: orbit: Target 360deg / 8s; }

Scene { sound: Idle; }    // eight cylinders, four firings a turn
```

V8 · .style

```
Crank            { color: #7d868f; material: brushed; }
.journal, .crankpin { color: #e3e8ee; material: chrome; }
Bore             { color: #a9cbd9; material: glass; opacity: 0.18; outline: 1px #5f8798; }
Piston           { color: #dde2e8; material: chrome; }
ConRod           { color: #a7b0ba; material: brushed; }
.head            { color: #4a5159; material: brushed; }
.cover           { color: #a3221a; material: brushed; }
Intake           { color: #cfd4da; material: chrome; }
Flywheel         { color: #5c646d; material: brushed; }
Pulley           { color: #30363d; material: metal; }
Camshaft, .lobe  { color: #c9a43a; material: gold; }
Alternator       { color: #8d969f; material: brushed; }
CL1              { trail: mid #e0a63a; }     // the path of one rod's midpoint
```

V8 · .scene

```
// A cross-plane V8, cut away to show its moving parts.
// Only the crankshaft is driven. Eight connecting rods and eight pistons are closed loops,
// solved every frame; a belt turns the camshaft at half speed and spins the alternator.
let r = 45mm;                       // crank throw: a 90mm stroke
let a1 = 90deg;  let a2 = 0deg;     // cross-plane crank: throws 90 degrees apart
let a3 = 180deg; let a4 = 270deg;
let left = 45deg; let right = -45deg;  // the two banks, 90 degrees apart

Crank::Group {
    main:  Cylinder.journal { radius: 26mm; height: 30mm; orient: 90deg about x; repeat: 5 step (0, 0, 110mm); at: (0, 0, -220mm); }
    pin1:  Cylinder.crankpin { radius: 22mm; height: 46mm; orient: 90deg about x; at: (r * cos(a1), r * sin(a1), -165mm); }
    pin2:  Cylinder.crankpin { radius: 22mm; height: 46mm; orient: 90deg about x; at: (r * cos(a2), r * sin(a2), -55mm); }
    pin3:  Cylinder.crankpin { radius: 22mm; height: 46mm; orient: 90deg about x; at: (r * cos(a3), r * sin(a3), 55mm); }
    pin4:  Cylinder.crankpin { radius: 22mm; height: 46mm; orient: 90deg about x; at: (r * cos(a4), r * sin(a4), 165mm); }
    web1:  Box.web { size: (58mm, 112mm, 12mm); orient: a1 - 90deg about z; repeat: 2 step (0, 0, 58mm); at: (10mm * cos(a1), 10mm *
    sin(a1), -194mm); }
    web2:  Box.web { size: (58mm, 112mm, 12mm); orient: a2 - 90deg about z; repeat: 2 step (0, 0, 58mm); at: (10mm * cos(a2), 10mm *
    sin(a2), -84mm); }
    web3:  Box.web { size: (58mm, 112mm, 12mm); orient: a3 - 90deg about z; repeat: 2 step (0, 0, 58mm); at: (10mm * cos(a3), 10mm *
    sin(a3), 26mm); }
    web4:  Box.web { size: (58mm, 112mm, 12mm); orient: a4 - 90deg about z; repeat: 2 step (0, 0, 58mm); at: (10mm * cos(a4), 10mm *
    sin(a4), 136mm); }
    nose:  Rod { length: 70mm; radius: 16mm; orient: 90deg about x; at: (0, 0, 270mm); }
    tail:  Rod { length: 40mm; radius: 30mm; orient: 90deg about x; at: (0, 0, -255mm); }
    anchor l1: (r * cos(a1), r * sin(a1), -177mm);   anchor r1: (r * cos(a1), r * sin(a1), -153mm);
    anchor l2: (r * cos(a2), r * sin(a2), -67mm);    anchor r2: (r * cos(a2), r * sin(a2), -43mm);
    anchor l3: (r * cos(a3), r * sin(a3), 43mm);     anchor r3: (r * cos(a3), r * sin(a3), 67mm);
    anchor l4: (r * cos(a4), r * sin(a4), 153mm);    anchor r4: (r * cos(a4), r * sin(a4), 177mm);
    at: (0, 0, 0);
}

// One line per cylinder, piston and rod, from three part types.
define Bore(row, a) :: Cylinder { radius: 50mm; height: 190mm; orient: a about z; at: (-185mm * sin(a), 185mm * cos(a), row); }
define Piston(bore) :: Cylinder { radius: 47mm; height: 58mm; slider: axis on bore; }
define ConRod(crank, piston, a) :: Link { length: 150mm; width: 20mm; thickness: 10mm; orient: 90deg + a about z; hinge: end.a on crank;
     pin: end.b to piston; }

L1::Bore(-177mm, left);  L2::Bore(-67mm, left);  L3::Bore(43mm, left);  L4::Bore(153mm, left);
R1::Bore(-153mm, right); R2::Bore(-43mm, right); R3::Bore(67mm, right); R4::Bore(177mm, right);
PL1::Piston(L1.axis); PL2::Piston(L2.axis); PL3::Piston(L3.axis); PL4::Piston(L4.axis);
PR1::Piston(R1.axis); PR2::Piston(R2.axis); PR3::Piston(R3.axis); PR4::Piston(R4.axis);
CL1::ConRod(Crank.l1, PL1.center, left);  CL2::ConRod(Crank.l2, PL2.center, left);
CL3::ConRod(Crank.l3, PL3.center, left);  CL4::ConRod(Crank.l4, PL4.center, left);
CR1::ConRod(Crank.r1, PR1.center, right); CR2::ConRod(Crank.r2, PR2.center, right);
CR3::ConRod(Crank.r3, PR3.center, right); CR4::ConRod(Crank.r4, PR4.center, right);

// Cylinder heads, valve covers and the intake between the banks.
HeadL::Box.head   { size: (150mm, 46mm, 460mm); orient: left about z; at: (-303mm * sin(left), 303mm * cos(left), 0); }
HeadR::Box.head   { size: (150mm, 46mm, 460mm); orient: right about z; at: (-303mm * sin(right), 303mm * cos(right), 0); }
CoverL::Box.cover { size: (118mm, 30mm, 430mm); orient: left about z; at: (-341mm * sin(left), 341mm * cos(left), 0); }
CoverR::Box.cover { size: (118mm, 30mm, 430mm); orient: right about z; at: (-341mm * sin(right), 341mm * cos(right), 0); }
Intake::Box       { size: (130mm, 54mm, 420mm); at: (0, 262mm, 0); }

// Flywheel at the back; pulleys and belts at the front.
Flywheel::Gear     { teeth: 96; module: 3mm; width: 18mm; attach: face.front to Crank.tail.end.a; }
CrankPulley::Pulley { radius: 40mm; width: 24mm; attach: face.back to Crank.nose.end.b; }
CamPulley::Pulley  { radius: 80mm; width: 24mm; at: (0, 150mm, 317mm); belt: CrankPulley; }
Camshaft::Rod      { length: 470mm; radius: 10mm; orient: 90deg about x; attach: end.b to CamPulley.face.back; }
Lobe::Box.lobe     { size: (16mm, 30mm, 14mm); repeat: 8; orient: i * 135deg about z;
                     attach: center to Camshaft.center + (-7mm * sin(i * 135deg), 7mm * cos(i * 135deg), -245mm + i * 50mm); }
AltPulley::Pulley  { radius: 26mm; width: 24mm; at: (-175mm, 40mm, 317mm); belt: CrankPulley; }
Alternator::Cylinder { radius: 44mm; height: 90mm; orient: 90deg about x; attach: face.top to AltPulley.face.back; }

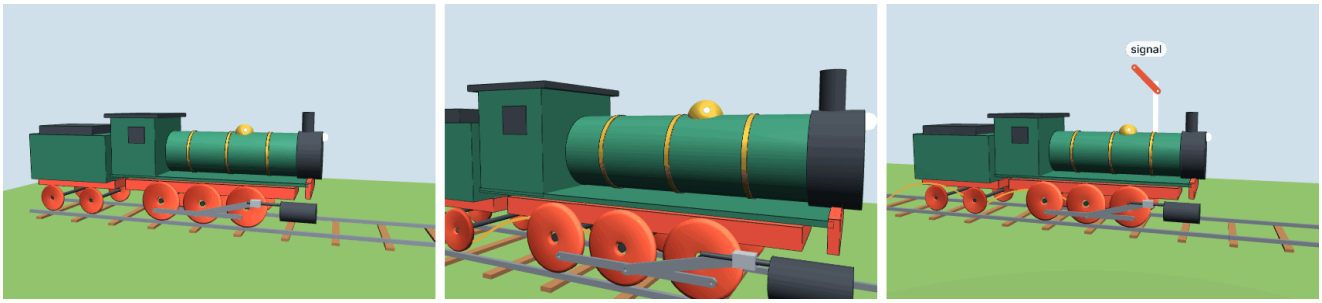
Target::Sphere { radius: 1mm; at: (0, 70mm, 0); visible: false; }

// sounds: generated by Orrery, free to use
Idle::Sound { kind: rumble; length: 8s; volume: 70%; }

Observer { position: (760mm, 560mm, 900mm); look-at: Target; }
Light { kind: directional; direction: (-0.5, -1, -0.6); power: 0.8; }
Light { kind: directional; direction: (0.9, -0.3, 0.5); power: 0.4; }
Scene { duration: 8s; background: #0b0f14; }
```

Steam locomotive: a second showcase

One speed is written for the whole scene. Six wheels roll on the track; the coupling rod and the main rod are two closed loops, solved at every frame; the crosshead slides wherever the main rod pushes it; the crank pin draws its curtate cycloid; a signal clears; and the camera pushes in on the valve gear and pulls back out. About 75 lines in all.



Three frames of the exported GIF, rendered with `orrery render`*: pulling away at 1 s, the push-in on the valve gear at 4.5 s, and the signal clearing at 9.3 s. The motion file comes first below: it is the whole of the animation.*

Locomotive · .motion

```
// The only speed in the whole scene. Everything else follows from it.
let cruise = (420mm/s, 0, 0);

Loco {
    0s..3s:  move: (0, 0, 0) -> cruise ease-in;      // pull away from a stand
    then 7s: move: cruise;
}

Driver, Small { forever: roll: Track; }

Arm { 8.3s..8.9s: rotate: 0deg -> -45deg ease-out; }    // the signal drops to clear the line

// Follow the engine, push in on the valve gear, then pull back out.
Observer {
    forever:   follow: Loco.view offset (260mm, 100mm, 900mm);
    2.5s..5s:  move: (0, 0, 0) -> (-40mm, -70mm, -470mm) ease-in-out;
    5.5s..8s:  move: (0, 0, 0) -> (40mm, 70mm, 470mm) ease-in-out;
}

// Steam: a hiss as it pulls away, then two puffs per turn of the 45mm driving wheels, faster as it speeds up
Loco {
    0s..1s:  sound: Steam;
    0.6s..2s: sound: Chuff every 0.6s;
    2s..3s:  sound: Chuff every 0.42s;
    3s..10s: sound: Chuff every 0.337s;    // 420mm/s on a 283mm wheel: 1.49 turns a second, two puffs each
}
Arm { 8.3s..8.9s: sound: Signal at end; }
```

Locomotive · .style

```
.engine        { color: #1f4e3d; material: plastic; outline: 1px #0c2018; }
.dark          { color: #24282d; material: matte; }
.red           { color: #a8382a; }
.trim          { color: #c9a43a; material: metal; }
Loco.dome      { color: #c9a43a; material: metal; }
Loco.lamp      { color: #fff4c2; luminosity: 90%; }
Loco.frame     { color: #a8382a; }
Rod            { color: graphite; material: metal; }
Driver         { color: gradient(#6e1d13, #c2492f, axis: x); material: plastic; }
Small          { color: gradient(#6e1d13, #c2492f, axis: x); }
Side, Main     { color: steel; material: metal; }
Cross          { color: #c7ccd2; material: metal; }
.rail          { color: #8f99a3; material: metal; }
.sleeper       { color: #6b4a2f; material: matte; }
Ground         { color: #4f6b3a; material: matte; }
Post           { color: #f2efe6; }
Arm            { color: #b8442e; label: "signal"; }
D2             { trail: crank 2.4s #e0a63a; }      // the crank pin traces a curtate cycloid
```

Locomotive · .scene

```
// A steam locomotive, built only from library shapes.
// Nothing here is animated by hand except the train's speed: the wheels roll, the coupling rod and
// main rod are solved as closed loops, and the crosshead slides wherever the main rod pushes it.
let r = 45mm;                    // driving wheel radius
let pitch = 100mm;               // axle spacing

Loco::Group.engine {
    frame:     Box { size: (400mm, 24mm, 96mm); at: (40mm, 30mm, 0); }
    footplate: Box { size: (450mm, 6mm, 156mm); at: (45mm, 55mm, 0); }
    boiler:    Cylinder { radius: 48mm; height: 300mm; orient: 90deg about z; at: (95mm, 108mm, 0); }
    band:      Cylinder.trim { radius: 49mm; height: 6mm; orient: 90deg about z; repeat: 3; at: (0mm + i * 90mm, 108mm, 0); }
    smokebox:  Cylinder.dark { radius: 52mm; height: 50mm; orient: 90deg about z; at: (268mm, 108mm, 0); }
    chimney:   Cylinder.dark { radius: 13mm; height: 50mm; at: (262mm, 176mm, 0); }
    dome:      Sphere { radius: 22mm; at: (120mm, 152mm, 0); }
    cab:       Box { size: (110mm, 135mm, 156mm); at: (-122mm, 122mm, 0); }
    window:    Box.dark { size: (40mm, 36mm, 158mm); at: (-112mm, 150mm, 0); }
    roof:      Box.dark { size: (134mm, 8mm, 170mm); at: (-122mm, 193mm, 0); }
    beam:      Box.red { size: (10mm, 32mm, 156mm); at: (270mm, 44mm, 0); }
    lamp:      Sphere { radius: 9mm; at: (296mm, 148mm, 0); }
    axle:      Rod { length: 132mm; radius: 5mm; orient: 90deg about x; repeat: 3; at: (-60mm + i * pitch, 0, 0); }
    cyl:       Cylinder.dark { radius: 20mm; height: 70mm; orient: 90deg about z; at: (252mm, 0, 84mm); }
    guide:     Rod { length: 72mm; radius: 2.5mm; orient: 90deg about z; at: (182mm, 13mm, 84mm); }
    anchor view: (60mm, 110mm, 0);           // where the camera looks
    at: (0, r, 0);
}

// Driving wheels: one line each. The crank anchor is where the rods attach.
define Driver(mount) :: Wheel { radius: r; width: 10mm; hinge: axis on mount; anchor crank: (0, -24mm, 0); }
D1::Driver(Loco.axle1.end.b);
D2::Driver(Loco.axle2.end.b);
D3::Driver(Loco.axle3.end.b);
D4::Driver(Loco.axle1.end.a);
D5::Driver(Loco.axle2.end.a);
D6::Driver(Loco.axle3.end.a);

// Valve gear. Two closed loops: the coupling rod joins all three cranks,
// and the main rod joins the middle crank to a crosshead on a slider.
Side::Link  { length: 200mm; width: 10mm; thickness: 4mm; hinge: end.a on D1.crank + (0, 0, 9mm);
              pin: end.b to D3.crank;  pin: mid to D2.crank; }
Main::Link  { length: 140mm; width: 12mm; thickness: 4mm; hinge: end.a on D2.crank + (0, 0, 15mm);
              pin: end.b to Cross.center; }
Cross::Box  { size: (14mm, 22mm, 12mm); orient: -90deg about z; slider: axis on Loco.guide.axis; }
PistonRod::Rod { length: 80mm; radius: 3mm; orient: -90deg about z; attach: end.a to Cross.center; }

// The tender rides behind, fused to the cab.
Tender::Group.engine {
    tank:  Box { size: (210mm, 110mm, 156mm); }
    coal:  Box.dark { size: (150mm, 20mm, 120mm); align: face.bottom; at: tank.face.top + (-15mm, 0, 0); }
    frame: Box.red { size: (220mm, 20mm, 96mm); align: face.top; at: tank.face.bottom; }
    axle:  Rod { length: 132mm; radius: 4mm; orient: 90deg about x; repeat: 2; at: (-55mm + i * 110mm, -103mm, 0); }
    attach: tank.face.right to Loco.cab.face.left + (-16mm, -30mm, 0);
}
define Small(mount) :: Wheel { radius: 32mm; width: 8mm; hinge: axis on mount; }
T1::Small(Tender.axle1.end.b);
T2::Small(Tender.axle2.end.b);
T3::Small(Tender.axle1.end.a);
T4::Small(Tender.axle2.end.a);

// The line: rails, sleepers, and a semaphore signal that clears as the train approaches.
Track::Group {
    railN:   Box.rail { size: (5000mm, 8mm, 6mm); at: (2100mm, -4mm, 65mm); }
    railS:   Box.rail { size: (5000mm, 8mm, 6mm); at: (2100mm, -4mm, -65mm); }
    sleeper: Box.sleeper { size: (16mm, 6mm, 190mm); repeat: 50; at: (-350mm + i * 100mm, -11mm, 0); }
}
Ground::Box { size: (5600mm, 4mm, 1400mm); at: (2100mm, -16mm, 0); }
Post::Box   { size: (12mm, 340mm, 12mm); at: (3250mm, 156mm, -190mm); }
Arm::Link   { length: 90mm; width: 16mm; thickness: 4mm; orient: 180deg about z; hinge: end.a on (3256mm, 300mm, -182mm); }

// sounds: generated by Orrery, free to use
Chuff::Sound  { kind: chug; volume: 80%; }
Steam::Sound  { kind: hiss; length: 1.4s; volume: 50%; }
Signal::Sound { kind: clang; pitch: 140%; volume: 45%; }

Observer { position: (260mm, 200mm, 900mm); }
Light { kind: directional; direction: (-0.4, -1, -0.7); power: 0.9; }
Scene { duration: 10s; background: #cfe0ea; }
```

Car: define, repeat, acceleration, roll and a following camera

Car · .scene

```
// A car built only from library shapes. No car primitive.
let wheel = 55mm;

Car::Group.paint {
    chassis: Box { size: (440mm, 50mm, 180mm); }
    cabin:   Box { size: (230mm, 80mm, 168mm); align: face.bottom; at: chassis.face.top + (-25mm, 0, 0); }
    glass:   Box { size: (234mm, 46mm, 172mm); at: cabin.center + (0, 10mm, 0); }
    hood:    Box { size: (120mm, 14mm, 176mm); align: face.bottom; at: chassis.face.top + (150mm, 0, 0); }
    bumper:  Box { size: (12mm, 18mm, 186mm); at: chassis.face.right + (4mm, -18mm, 0); }
    lamp:    Sphere.lamp { radius: 12mm; repeat: 2; at: chassis.face.right + (0, 6mm, 58mm - i * 116mm); }
    tail:    Box.tail { size: (6mm, 14mm, 30mm); align: face.right; repeat: 2; at: chassis.face.left + (0, 8mm, 62mm - i * 124mm); }
    axle:    Rod { length: 230mm; radius: 6mm; orient: 90deg about x; repeat: 2; at: chassis.face.bottom + (140mm - i * 280mm, 10mm, 0);
     }
    at: (0, 74mm, 0);
}

// A reusable part type: one line per wheel. The hinge rides on the car.
define Tyre(mount) :: Wheel { radius: wheel; width: 34mm; hinge: axis on mount; }
FL::Tyre(Car.axle1.end.b);
FR::Tyre(Car.axle1.end.a);
RL::Tyre(Car.axle2.end.b);
RR::Tyre(Car.axle2.end.a);

Road::Group {
    surface: Box { size: (6000mm, 4mm, 700mm); }
    dash: Box.dash { size: (220mm, 2mm, 18mm); at: surface.face.top + (-2500mm, 1mm, 0); repeat: 11 step (500mm, 0, 0); }
    at: (1500mm, 2mm, -60mm);
}

// sounds: generated by Orrery, free to use
Motor::Sound { kind: whirr; pitch: 70%; length: 4.2s; volume: 40%; }
Brake::Sound { kind: squeak; pitch: 85%; length: 1s; volume: 35%; }

Observer { position: (380mm, 330mm, 1150mm); }
Light { kind: directional; direction: (-0.5, -1, -0.6); power: 0.9; }
Scene { duration: 5s; }
```

Car · .style

```
.paint      { color: #2f6f8f; material: plastic; outline: 1px #13303d; }
Car.glass   { color: #a9d4e0; material: glass; }
.lamp       { color: #fff4c2; material: plastic; }
.tail       { color: #d23b2e; }
Car.bumper  { color: graphite; }
Rod         { color: graphite; material: metal; }
Tyre        { color: gradient(#16181b, #5d6670, axis: x); material: matte; }
Road        { color: #3b4148; material: matte; }
.dash       { color: #f2efe6; }
```

Car · .motion

```
// A change between two speeds is an acceleration. Units decide: mm/s is a speed.
let cruise = (600mm/s, 0, 0);

Car {
    0s..1.5s: move: (0, 0, 0) -> cruise ease-in;     // pull away
    then:     move: cruise;                          // cruise for 1.5 s
    3s..4.2s: move: cruise -> (0, 0, 0) ease-out;    // brake to a stop
}

// Every Tyre rolls: the spin always matches the car, even while it speeds up and slows down.
Tyre { forever: roll: Road; }

Observer { forever: follow: Car offset (380mm, 260mm, 1150mm); }

Scene { sound: Motor; }
Car { 3s..4.2s: sound: Brake; }
```

Engine: a slider-crank driven by a flywheel, with a trail

Engine · .scene

```
// A single-cylinder engine: a slider-crank. Only the flywheel turns.
// The connecting rod's pin keeps it on the piston, so the piston works out its own position.
Flywheel::Disc { radius: 70mm; thickness: 10mm; at: (0, 0, -20mm); }
Weight::Box    { size: (60mm, 26mm, 10mm); attach: Flywheel.face.front + (0, -40mm, 5mm); }
Crank::Link    { length: 40mm; width: 20mm; orient: 90deg about z; attach: end.a to Flywheel.face.front + (0, 0, 10mm); }

Bore::Cylinder   { radius: 34mm; height: 180mm; at: (0, 180mm, 0); }
Piston::Cylinder { radius: 31mm; height: 44mm; slider: axis on Bore.axis; }
Rod::Link        { length: 150mm; width: 16mm; orient: 90deg about z; hinge: end.a on Crank.end.b + (0, 0, 8mm); pin: end.b to Piston.
    center; }

// sounds: generated by Orrery, free to use
Putt::Sound { kind: chug; pitch: 70%; }

Observer { position: (120mm, 150mm, 640mm); look-at: (0, 110mm, 0); }
Light { kind: directional; direction: (-0.5, -0.7, -1); power: 0.85; }
Scene { duration: 3s; }
```

Engine · .style

```
Flywheel { color: gradient(#3b4148, #8f99a3, axis: x); material: metal; }
Weight   { color: #3b4148; }
Crank    { color: #b8442e; }
Rod      { color: steel; material: metal; trail: mid #e0a63a; }
Piston   { color: #c7ccd2; material: metal; label: "piston"; }
Bore     { color: glass; material: glass; outline: 1px #5d8a96; }
```

Engine · .motion

```
Flywheel { forever: rotate: 240deg/s; }

// one firing every turn of the flywheel (240deg/s: one turn per 1.5 s)
Flywheel { forever: sound: Putt every 1.5s; }
```

Windscreen wiper: a four-bar with the blade fixed to the rocker

Wiper · .scene

```
// A windscreen wiper. A motor crank drives a four-bar linkage and the arm is fixed to the
// rocker, so each full turn of the motor becomes one sweep there and back.
define Bar(len) :: Link { length: len; width: 10mm; }

Screen::Box { size: (520mm, 260mm, 4mm); at: (0, 110mm, -30mm); }
Motor::Cylinder { radius: 18mm; height: 20mm; orient: 90deg about x; at: (-150mm, -70mm, -12mm); }

Crank::Bar(30mm)  { hinge: end.a on (-150mm, -70mm, 0); }
Drive::Bar(152mm) { hinge: end.a on Crank.end.b + (0, 0, 5mm); pin: end.b to Lever.end.b; }
Lever::Bar(40mm)  { orient: -90deg about z; hinge: end.a on (0, -20mm, 0); }
Arm::Rod   { length: 210mm; radius: 3mm; attach: end.a to Lever.end.a + (0, 0, 12mm); }
Blade::Box { size: (12mm, 150mm, 8mm); attach: face.top to Arm.end.b + (0, 0, 6mm); }

// sounds: generated by Orrery, free to use
Squeak::Sound { kind: squeak; length: 0.8s; volume: 45%; }

Observer { position: (0, 60mm, 820mm); look-at: (0, 70mm, 0); }
Light { kind: directional; direction: (-0.3, -0.6, -1); power: 0.85; }
Scene { duration: 4s; }
```

Wiper · .style

```
Screen { color: #a9d4e0; material: glass; }
Motor  { color: #3b4148; }
Bar    { color: #6b7580; material: plastic; }
Crank  { color: #e0a63a; label: "motor crank"; }
Arm    { color: graphite; }
Blade  { color: #b8442e; trail: face.top 1s #2f6f8f; }
```

Wiper · .motion

```
Crank { forever: rotate: 180deg/s; }

// one squeak per sweep: the crank turns once in 2 s, the blade sweeps twice
Crank { forever: sound: Squeak every 1s; }
```

12 Diagnostics

Every problem names its file, line and column, says what went wrong and suggests a fix. Errors stop the preview from updating; warnings do not. **Copy as JSON** in the studio, check `--json` on the command line and the MCP check tool all give the same format:

Diagnostic JSON

```
{ "severity": "error", "code": "E0353", "file": "scene", "line": 3, "col": 5,
  "message": "The linkage locks at 0.47s: Coupler.end.b and Rocker.end.b can no longer meet.",
  "fix": "The driven joint asks for a position the links cannot reach. Shorten the crank, ..." }
```

Codes	Area	Typical causes
E0100–E0122	Syntax, let, repeat	A missing ; or }, an unknown unit, a let name with a capital, a let defined twice, repeat outside 1–200
E0111	Older syntax	mesh G1 G2; or upto 10s:. The message shows the current form.
E0130–E0139	define	Lower-case or built-in type name, unknown or missing parameter, too many values, group in a group, a type built on itself
E0201–E0215	Parts	Unknown type, a required property missing, under 6 teeth, a name used twice, orient without about
E0300–E0311	Values, anchors	A vector where a number was expected; an anchor the shape does not have
E0320	Names	Misspelled, or declared further down
E0330–E0345	attach, hinge, slider	Attached to itself; neither side a line; a hinge between attached parts
E0350–E0354	pin, loops	Badly written pin; loop cannot close; linkage locks; every joint in the loop driven
E0400	Conflicts	One part driven twice (a motion rule and a hinge, two meshes)
E0410–E0419	mesh, couple, belt	Not a gear, different modules, wrong distance, ratios that disagree, no axis
E0420	Ranges	A hinge rotation or slider move outside its range
E0500–E0541	Motion	Unknown target, a range that ends before it starts, bad rotate/move/follow/orbit forms, a speed change in forever
E0600–E0606	Style	Unknown colour or material, one-colour gradient, a trail value that is neither anchor nor colour
E0998	Simulation	A problem that only appears while the animation plays
W0202–W0602	Warnings	Ignored property, a mesh nothing turns, a selector or class that matches nothing, a non-camera property

13 Tools for people and AI

Studio

- **Editor and Problems** on the left; click a problem to jump to it. **Preview** on the right; click a part to jump to its code.
- **AI prompt** copies the syntax as instructions any chat assistant can follow (appendix B), plus what you type and, if ticked, your current code and the state at the current time. In the Split view it asks for three code blocks; in the Combined view, for one block with @scene, @style and @motion sections.
- **Guided tour** (More › Take the tour): dims the studio and spotlights one feature at a time, with Previous, Next and Skip.
- **Split / Combined** switches between three tabs and one file with @scene, @style and @motion sections. Pasting a single-file project into the Split view fills all three tabs.
- **Layout**: drag the divider between code and preview, or above Problems; double-click to reset. The code panel can sit left, right, top or bottom, and either side can be expanded to the full window (Esc restores).
- **Type hints** suggest shape types, properties (required ones marked), anchors after Part., keywords, units, materials, colours, easings and time blocks as you type, with the property's form on the bottom line. Ctrl+Space opens them; Settings can make them manual or turn them off.
- **Header**: Examples (a dialog with a picture of each; each opens in a new tab), Split / Combined, AI prompt, Save, Export and your account; open, 3D models, guide, settings, tour and cookie preferences sit under More (⋯). **Accounts** (Google sign-in) are needed to download exports when the studio runs on the Orrery server.
- **Settings** (More › Settings, or Ctrl+,): theme, accent, syntax colours, interface size; editor font, size, spacing, ligatures, indent, line numbers, current-line highlight, bracket closing, auto-indent, type hints; preview update timing (or manual with Ctrl+Enter), autoplay, loop, speed, joint axes and gear lines, floor grid, labels, trails, sharpness, background; layout and export defaults.
- **Export**: MP4 or WebM video up to 1920 px and 60 fps, GIF up to 800 px, PNG frames (.zip, with transparency) or a 3D player (.html). **Save** (Ctrl+S) writes .orrery or .zip; Save as adds three loose files and JSON. **Copy state** (timeline, or right-click the preview) copies every part's position, rotation and box at the current moment, for an AI.

Command line (cli folder, Node 18 or later)

Command	Does
`check proj/ --json`	Compiles and lists problems; exit code 1 on errors
`sample proj/ --at 0,2 --points Crank.end.b`	Prints positions, rotations and anchor points as JSON
`render proj/ -o out.gif --width 480 --fps 20`	Renders a GIF in a headless browser
`eval tasks.json answers/`	Scores AI answers against the 12-task test set
`prompt a crane lifting a box`	Prints the AI prompt with your task
`examples out/ [--single]`	Writes every studio example as a project folder, or as one .orrery file each
`join proj/ -o project.orrery`	Combines three files into one single-file project
`split project.orrery out/`	Splits a single-file project into three files
`mcp`	MCP server: check, sample, render, prompt, spec

A project is a folder with .scene, .style and .motion files, a single .orrery file, a .orrery.json file, or a .scene file whose siblings share its name. MCP tools take either project (one file) or scene, style and motion.

A Quick reference card

Declare

```
let r = 40mm;
Name::Type.cls { p: v; }
Name::Group { m: Box { } }
anchor tip: (0, 10mm, 0);
define T(a, b = 1mm) :: Box { }
X::T(5mm) { at: (0,0,0); }
repeat: 4 step (10mm,0,0);
```

Place

```
at: (x, y, z);
at: Other.face.top + (0,5mm,0);
align: face.bottom;
orient: 90deg about x;
```

Connect

```
attach: end.a to B.face.top;
hinge: edge.left on F.edge.right;
hinge: end.a on (0, 0, 0);
slider: axis on Bore.axis;
range: -45deg..45deg;
mesh: G1;   mesh: G1 at 60deg;
couple: Hub 1/12;
belt: Motor;
pin: end.b to Rocker.end.b;
```

Scene

```
Observer { position: (..);
           look-at: Part; }
Light { kind: directional;
        direction: (-1,-2,-1); }
Scene { duration: 4s; }
```

Units

```
mm cm m    mm/s    mm/s2
deg rad turn deg/s deg/s2
s ms   %   360deg / 12s
```

Shapes

```
Box size   Sphere radius
Cylinder radius height
Cone radius height
Torus radius tube
Rod length radius
Link length width thickness
Gear teeth module width
Pulley/Wheel radius width
Disc radius thickness
Spring radius length turns
Line/Arrow from to stroke
Curve domain point stroke
Tube Capsule Prism Pyramid
Dome Wedge Plate Beam Ring
Nut Bolt Cam Star Mesh
2D: Triangle Ellipse Polygon
Sector Arc Path
```

Anchors

```
center  com  com.z
face.top/bottom/left/
     right/front/back
edge.top  edge.left-top
corner.left-top-front
axis  end.a  end.b  mid
pole.top  apex  tip (custom)
```

Motion blocks

```
T { forever: p: v; }
T { 0s..2s: p: v;
    then: p: v;
    then 3s: p: v; }
.cls, Tyre, Observer
```

Values

```
90deg          set
0deg -> 90deg  change
60deg/s        speed
(0,0,0) -> (v) speed change
(0,-9.81m/s2,0) accel
20deg*sin(t*3) function
```

Properties

```
rotate: 60deg/s about x;
orbit: Sun 30deg/s;
move: (500mm/s, 0, 0);
along: Path 12s;
x: 0;  opacity: 1 -> 0;
rotate: 0deg -> 90deg;
roll: Road;
move: -90mm -> 90mm;
```

Camera

```
follow: Car offset (x,y,z);
orbit: Part 60deg/s;
look-at: Part;  zoom: 2;
```

Sound

```
Knock::Sound { kind: knock; }
sound: Knock at end;
sound: Tick every 1s 50%;
Scene { sound: Bell at 4s; }
```

Style

```
color: #hex | steel |
  rgba(r,g,b,a) | transparent |
  gradient(#a, #b, axis: x);
material: metal;
outline: 1px #222;
opacity: 0.5; label: "A";
luminosity: 80%;
trail: tip 1.5s #c2410c;
```

B The AI prompt

This is the text the studio's **AI prompt** *button copies, followed by your task (and your current code, if you tick that box). This is the three-file version; in the one-file view the OUTPUT paragraph asks for a single block with @scene, @style and @motion sections instead. Print either with* `orrery prompt` *(add* `--single`*). It runs to about 1,600 words.*

```
You write animations in MotionScript 0.6, a small CSS-like language for mechanism animations. A project has three
parts:
scene (what exists), style (how it looks) and motion (how it moves). Write relations between parts, not raw
coordinates, wherever you can.

RULES
- Every property ends with a semicolon. Comments are // to the end of the line, or /* ... */ (as in CSS). Never use #
for comments: # starts a colour or an id.
- Part names start with a capital letter (Crank). let names and define parameters are lower case.
- Numbers carry units with no space: 40mm, 90deg, 2s. Axes: x right, y up, z towards the viewer. A vector is (x, y,
z).
- Units decide meaning: mm, cm, m = position or length; mm/s = speed; mm/s2 = acceleration; deg, rad, turn = angle;
deg/s = spin rate; deg/s2 = angular acceleration. "360deg / 12s" is also a rate.
- Functions: sin cos tan sqrt abs min max pow floor exp atan2 length, and PI. t is time in seconds inside motion
values.
- A project can also be ONE file: a line @scene starts the scene, @style the style, @motion the motion.

SCENE
let r = 40mm;
Name::Type.optionalClass { property: value; }
Name::Group { member: Box { ... }  anchor tip: (0, 10mm, 0);  at: (0, 0, 0); }
define Tyre(mount, r = 55mm) :: Wheel { radius: r; width: 30mm; hinge: axis on mount; }    // reusable part type
FL::Tyre(Car.axle1.end.b);                                                                  // values by position or
name
Observer { position: (x, y, z); look-at: Part; }   Light { kind: directional; direction: (-1, -2, -1); power: 0.85; }
Scene { duration: 4s; }      (optional: background: #070a10 | transparent; grid: floor;)
Placement: at: point; align: face.bottom; (puts that anchor on the at point) orient: 90deg about x;
repeat: 4 step (100mm, 0, 0); makes Name1..Name4; i (0, 1, 2 ...) works in expressions. Points can be anchor maths:
Frame.face.top + (0, 10mm, 0).

SHAPES (property list -> anchors; every shape and group also has center, and com: its true centre of mass as a point,
with com.x, com.y, com.z as lines through it, e.g. hinge: com.z on Base.axis to pivot a part at its balance point.
center-of-mass is the long name for com.)
Box size (x, y, z) -> face.top/bottom/left/right/front/back, edge.<face>, edge.<f1>-<f2>, corner.<x>-<y>-<z>, axis (y)
Sphere radius -> pole.top, pole.bottom, axis (y)      Cylinder radius, height -> axis (y), face.top, face.bottom
Cone radius, height -> axis, face.top, face.bottom, apex      Torus radius, tube -> axis (z)
Rod length, radius -> axis (y), end.a (bottom), end.b (top)
Link length, width, thickness -> end.a, end.b, mid, axis: all lines along z (pin axles). A Link lies along x. Use it
for linkages.
Gear teeth, module (default 2mm), width -> axis (z), face.front, face.back      Pulley, Wheel radius, width -> axis (
z), face.front, face.back
Disc radius, thickness -> axis (z), face.front, face.back      Spring radius, length, turns, wire -> axis (y), end.a,
end.b
Line, Arrow from, to, stroke -> start, end, mid      Curve domain: u in 0..1; point: (expression of u and t); stroke -
> start, mid, end
Tube radius, inner, height; Prism sides, radius, height; Pyramid sides, radius, height -> axis (y), face.top, face.
bottom (+ apex)
Capsule radius, length; Bolt radius, length, head -> axis (y), end.a, end.b (Bolt: head at end.b)      Dome radius ->
axis, face.bottom, pole.top
Ring radius, inner, thickness; Nut radius, thickness, hole; Star points, radius, inner, thickness; Cam radius, lift,
width -> axis (z), face.front, face.back (Cam: nose)
Wedge size (x, y, z); Plate size (w, h), thickness, corner, hole; Beam length, height, width, thickness (an I-beam
along x) -> like Box (Beam: end.a, end.b)
Mesh src: "file.stl" or "file.glb"; size (x, y, z); fit: contain|stretch -> like Box. Only for model files the user
has added to the project.
2D shapes lie in the x-y plane facing +z (thickness defaults to 1mm): Triangle size (w, h), apex; Ellipse size (w, h)
-> like Box + normal
Polygon sides, radius; Sector radius, angle, start, inner (pie slice or ring segment; angles anticlockwise from +x);
Path points: (x, y) (x, y) ... -> axis (z), normal, face.front, face.back
Arc radius, angle, start, stroke -> a curved line: start, mid, end, axis. For a flat, head-on view use Observer {
projection: orthographic; } looking along -z.
Group: members share a frame; refer to Car.axle1 and Car.axle1.end.b.

CONNECTIONS (written inside the moving part's block)
attach: own-anchor to Other.anchor;  or  attach: Other.anchor;      -> fused, moves as one
hinge: own-line on Other.line;  or  hinge: Other.line;  or  hinge: end.a on (0, 0, 0);      -> turns about the line
and rides on Other
slider: axis on Other.axis;      range: -45deg..45deg;  (limits a hinge or slider)
mesh: G1;  or  mesh: G1 at 60deg;      -> places the gear and meshes it, ratio -teeth1/teeth2
couple: Hub 1/12;      belt: Motor;      pin: own-point to Other.point;  (keeps two points together and closes a loop)
Closed loops: drive one joint; hinges and sliders in the loop with no motion rule are solved every frame. Four-bar:
```

````
  Crank::Link   { length: 40mm;  hinge: end.a on (0, 0, 0); }
  Coupler::Link { length: 120mm; hinge: end.a on Crank.end.b + (0, 0, 6mm); pin: end.b to Rocker.end.b; }
  Rocker::Link  { length: 90mm;  orient: 90deg about z; hinge: end.a on (110mm, 0, 0); }
  motion: Crank { forever: rotate: 180deg/s; }

STYLE (CSS-like selectors: Type or define name, Name, Name.member, .class, #Name, comma lists; later rules win)
color: #hex | #rrggbbaa | rgb(r, g, b) | rgba(r, g, b, a) | hsl(h, s%, l%) | transparent | CSS name | steel brass oak
copper aluminium iron rubber walnut glass graphite | gradient(#a, #b, axis: x);
material: matte | plastic | metal | brushed | chrome | gold | glass | flat | glow;   outline: 1px #2b3138;   opacity:
0.5 (or 50%; faces, outline and label fade; colour alpha multiplies faces only);   label: "text";
trail: [anchor] [time] [colour];  e.g. trail: tip 1.5s #c2410c;  draws the path a point follows

MOTION
Target, .class, DefineName { forever: prop: value;   0s..2s: prop: value;   then: prop: value;   then 3s: prop: value;
 }
Position or angle (90deg): holds from its block start; the latest started block wins.  Change (0deg -> 90deg ease-in-
out): moves across its block, then holds.
Speed (60deg/s, (500mm/s, 0, 0)): moves during its block, and what moved stays moved.  Change of speed ((0,0,0) -> (
600mm/s,0,0) ease-in): needs a time range.
Acceleration ((0, -9.81m/s2, 0)): constant within its block.  Any expression of t also works: 30deg * sin(t * 3.1416).
Easings: linear ease ease-in ease-out ease-in-out cubic-bezier(a, b, c, d).
Free parts: rotate: 60deg/s [about x|y|z|Part.axis]; orbit: Center 30deg/s; move: vector; (several move lines add up)
along: CurveName 12s; x: / y: / z:; opacity: (0s..1s: opacity: 0 -> 1 ease-in; fades in, a group fades all its
members)
Hinged parts: rotate: (an angle like 0deg -> 90deg, or a rate) and roll: Ground; (spin = distance / radius)
Sliders: move: -90mm -> 90mm;
Observer: follow: Part offset (x, y, z); orbit: Part 60deg/s; look-at: Part; zoom: 2; move: vector; position: (x, y,
z);
Drive only what starts the motion; everything connected follows.
SOUND: scene: Knock::Sound { kind: knock; } (kinds: tick tock ratchet clank latch gear spring rattle footstep clang
chug hiss rumble knock thud click snap pop tap whoosh swish whirr servo slide creak squeak chime bell ding beep blip
success alert hum; optional pitch: 120%; volume: 60%; length: 2s;)
  motion, in any rule: sound: Knock;  sound: Knock at end; (a part landing)  sound: Tick every 1s;  sound: Bell at 2s
60%;  Scene { sound: Bell at 4s; } for set times

AVOID
orient: 90deg; (needs "about x")  -  a top-level "mesh G1 G2;" (write mesh: G1; inside G2)  -  "upto 10s:" (write 0s..
10s:)
rotate: or offset: (they are now rotate: and move:)  -  rotate about ... on a hinged part  -  driving one part twice (
a motion rule plus a hinge or mesh)
hinging a Rod on a face anchor for a pendulum (face anchors are lines along their normal; use a Link, a Box edge or a
fixed point)

OUTPUT: reply with exactly three fenced code blocks labelled scene, style and motion, in that order, then one or two
sentences on what you built. Example of the shape of a reply:
```scene
G1::Gear { teeth: 36; at: (0, 0, 0); }
G2::Gear { teeth: 12; mesh: G1; }
```
```style
Gear { color: steel; material: metal; }
```
```motion
G1 { forever: rotate: 30deg/s; }
```
````